

Today 8:04 PM

SE LAB 4 – VIBE CODING

Maze Runner

Name: Thrupthi G D

SRN: AM476

Course: B.Tech CSE-AIML

GitHub Repository:

https://github.com/Thrupthi-AM476/12_maze_runner Objective:
Use Vibe Coding tools to fix the broken code and add the required features to the Maze Runner project.

Got it. This is your **SE Lab 4 – Vibe Coding: Maze Runner** lab, and you're preparing the final submission around the GitHub repository.

I have the basic details:

- **Name:** Thrupthi G D
- **SRN:** AM476
- **Course:** B.Tech CSE-AIML
- **Lab:** SE Lab 4 – Vibe Coding
- **Project:** Maze Runner
- **Objective:** Use Vibe Coding tools to fix the broken code and add the required features.
- **Repository:** [Maze Runner GitHub Repository ↗](#)

If you want, send me the **complete lab question/instructions (PDF, screenshot, or text)** and I can prepare the exact final submission structure for you, including:

1. What was broken initially
2. What you fixed using Vibe Coding
3. Features added
4. Prompts/chats used with the Vibe Coding tool
5. Before/after evidence
6. GitHub README content
7. Screenshots you should include
8. Final lab report/PDF content
9. How the repository should be organized
10. A final checklist before submission

That way we can make sure your submission matches the lab requirements exactly rather than adding unnecessary material.

TASK 1 : Add a BFS solver to my pygame maze. When H is pressed, highlight the shortest path from start to exit by drawing colored squares on each cell in the path. Toggle it off on second press.

After it works

Test:

- Start game
- Press H
- Path appears
- Press H again
- Path disappears
- Generate another maze with R
- Press H
- New path is calculated

Then:

```
git add .  
git commit -m "Add BFS shortest path hint"  
git push
```

For Task 1, you need to implement exactly this behavior in your Maze Runner project.

What needs to be added

1. BFS shortest-path solver

- Start from the maze's start cell.
- Explore neighboring cells using **Breadth-First Search**.
- Stop when the exit is reached.
- Reconstruct the shortest path.

2. H key

- First press → calculate and display the shortest path.
- Second press → hide the path.
- It should act as a toggle.

3. Path visualization

- Draw a colored square/overlay on every cell belonging to the path.
- Keep the player, maze walls, start, and exit visible as appropriate.

4. R key compatibility

- When a new maze is generated with R , the old BFS path must be cleared.

- Pressing H afterward must calculate the path for the **new maze**, not reuse the old one.

Required testing

Your demonstration should follow this exact sequence:

Plain text

```
Start game
↓
Press H
↓
Shortest path appears
↓
Press H
↓
Path disappears
↓
Press R
↓
New maze generated
↓
Press H
↓
New shortest path appears
```

Git commands after testing

Run these from your repository folder:

Bash

```
git add .
git commit -m "Add BFS shortest path hint"
git push
```

For your lab evidence

Take screenshots showing at least:

- Maze normally running
- Maze with BFS path after pressing H
- Path disappearing after pressing H again
- New maze after pressing R
- BFS path for the new maze
- Git commit/push successfully completed

If you're using a Vibe Coding tool such as Kiro/Claude Code, **the prompt you give it is also important evidence** for this lab because the objective specifically says to use Vibe Coding.

[give codes for it](#)

Yes. Since your Maze Runner already has a `main.py`, the safest way is to add BFS **without changing your existing maze-generation/game logic**.

Below is a clean implementation you can integrate.

1. Add these variables near your other game variables

`</>` Python

```
show_hint = False
shortest_path = []
```

2. Add the BFS function

Put this above your main game loop:

`</>` Python

```
from collections import deque

def bfs_shortest_path(maze, start, end):
    rows = len(maze)
    cols = len(maze[0])

    queue = deque([start])
    parent = {start: None}

    directions = [
        (-1, 0),
        (1, 0),
        (0, -1),
        (0, 1)
    ]

    while queue:
        current = queue.popleft()
```

Important: Here I assumed:

`</>` Python

```
0 = walkable cell
1 = wall
```

If your existing maze uses the opposite convention, change:

</> Python

```
maze[nr][nc] == 0
```

to:

</> Python

```
maze[nr][nc] == 1
```

3. Add the H key handling

Inside your existing Pygame event loop, add:

</> Python

```
if event.type == pygame.KEYDOWN:
    if event.key == pygame.K_h:
        show_hint = not show_hint

    if show_hint:
        shortest_path = bfs_shortest_path(maze, start, exit)
    else:
        shortest_path = []
```

So it behaves like:

</> Plain text

```
H → show path
H → hide path
H → show path
H → hide path
```

4. Clear the path when R generates a new maze

Find your existing R key code.

It may look something like:

</> Python

```
if event.key == pygame.K_r:
    maze = generate_maze()
```

Change it to:

</> Python

```
if event.key == pygame.K_r:
    maze = generate_maze()
    show_hint = False
    shortest_path = []
```

This is important because the old path should **not** remain after generating a new maze.

5. Draw the BFS path

After you draw your maze cells/walls but **before drawing the player**, add:

</> Python

```
if show_hint:
    for row, col in shortest_path:
        pygame.draw.rect(
            screen,
            (255, 215, 0),
            (
                col * CELL_SIZE,
                row * CELL_SIZE,
                CELL_SIZE,
                CELL_SIZE
            )
        )
```

This will make the shortest path appear as yellow squares.

If your cell size variable has a different name, such as:

</> Python

```
TILE_SIZE
```

use that instead of `CELL_SIZE` .

Complete BFS portion

Your final added code should essentially look like this:

</> Python

```
from collections import deque

show_hint = False
shortest_path = []

def bfs_shortest_path(maze, start, end):
    rows = len(maze)
    cols = len(maze[0])

    queue = deque([start])
    parent = {start: None}

    directions = [
        (-1, 0),
        (1, 0),
        (0, -1),
        (0, 1)
    ]
```

Then inside the event loop:

</> Python

```
if event.type == pygame.KEYDOWN:

    if event.key == pygame.K_h:
        show_hint = not show_hint

        if show_hint:
            shortest_path = bfs_shortest_path(maze, start, exit)
        else:
            shortest_path = []

    elif event.key == pygame.K_r:
        maze = generate_maze()
        show_hint = False
        shortest_path = []
```

And in the drawing section:

</> Python

```

if show_hint:
    for row, col in shortest_path:
        pygame.draw.rect(
            screen,
            (255, 215, 0),
            (
                col * CELL_SIZE,
                row * CELL_SIZE,
                CELL_SIZE,
                CELL_SIZE
            )
        )
    )

```

One important thing

Because I don't have your **actual main.py code in this message**, I don't want you to blindly paste this and accidentally use the wrong variable names (maze , start , exit , CELL_SIZE , etc.).

If you **upload your current main.py here**, I'll modify **your exact code** and give you the complete final main.py ready to replace the existing one, while keeping all your existing functionality unchanged.

TASK 2 — Fog of War Draw a dark overlay over the whole maze surface, then cut out a circular transparent region around the player using `pygame.draw.circle` on a surface with `SRCALPHA`.
 ### Prompt

Use:

Add fog of war to my maze game. Draw a dark overlay over the whole maze surface, then cut out a circular transparent region around the player using `pygame.draw.circle` on a surface with `SRCALPHA`. Reveal only nearby cells.

Test it properly

Move around the maze.

Check that:

- nearby cells are visible
- distant cells are hidden
- fog moves with the player

- player remains visible
- maze movement still works
- H still works
- R still generates a new maze

Then:

```
```bash
git add .
git commit -m "Add fog of war"
git push
```

Yes. For **Task 2**, add the fog as a separate transparent overlay so it doesn't interfere with your existing maze, BFS H , or R logic.

Assuming your existing variables are `screen` , `player` , `CELL_SIZE` , `maze_width` , and `maze_height` , use this.

## 1. Add these variables

Near your other game constants:

</> Python

```
FOG_RADIUS = 120
FOG_COLOR = (0, 0, 0, 220)
```

You can increase `FOG_RADIUS` if the visible area is too small.

## 2. Create the fog function

Add this above your main game loop:

</> Python

```
def draw_fog_of_war(screen, player_x, player_y):
 fog = pygame.Surface(screen.get_size(), pygame.SRCALPHA)

 fog.fill(FOG_COLOR)

 pygame.draw.circle(
 fog,
 (0, 0, 0, 0),
 (player_x, player_y),
```

```
 FOG_RADIUS
)

 screen.blit(fog, (0, 0))
```

The important part is:

Python

```
pygame.Surface(screen.get_size(), pygame.SRCALPHA)
```

This creates a surface that supports transparency.

Then:

Python

```
pygame.draw.circle(
 fog,
 (0, 0, 0, 0),
 (player_x, player_y),
 FOG_RADIUS
)
```

cuts out the transparent circular region around the player.

### 3. Call the fog function at the correct place

Your drawing order should be approximately:

Python

```
Draw maze
draw_maze()

Draw BFS path if H is enabled
if show_hint:
 for row, col in shortest_path:
 pygame.draw.rect(
 screen,
 (255, 215, 0),
 (
 col * CELL_SIZE,
 row * CELL_SIZE,
 CELL_SIZE,
 CELL_SIZE
```

```
)
)

Draw fog
draw_fog_of_war(screen, player_x, player_y)
```

## Very important

**Draw the player after the fog.**

Otherwise the fog can cover the player.

If your player is stored as a rectangle, you can do:

**</> Python**

```
draw_fog_of_war(
 screen,
 player_rect.centerx,
 player_rect.centery
)
```

and then draw the player.

---

## If your player uses row/column coordinates

For example, if you have:

**</> Python**

```
player_row
player_col
```

then convert them to screen coordinates:

**</> Python**

```
player_x = player_col * CELL_SIZE + CELL_SIZE // 2
player_y = player_row * CELL_SIZE + CELL_SIZE // 2

draw_fog_of_war(screen, player_x, player_y)
```

---

## Recommended final drawing order

## `</>` Python

## # 2. Maze

</> Plain text

FOG

</> Python

12/60

```
 if show_hint:
 shortest_path = bfs_shortest_path(maze, start, exit)
 else:
 shortest_path = []

elif event.key == pygame.K_r:
 maze = generate_maze()
 show_hint = False
 shortest_path = []
```

Fog of war is purely a **drawing layer**, so:

- H → BFS path still works
- H again → path disappears
- R → new maze
- Player movement → unchanged

## Test exactly this

### Plain text

```
Run game
↓
Move player
↓
Nearby maze visible
Distant maze hidden
↓
Move again
↓
Fog follows player
↓
Press H
↓
BFS path appears in visible area
↓
Press H
↓
BFS path disappears
↓
Press R
```

Then commit:

### Bash

```
git add .
git commit -m "Add fog of war"
```

```
git push
```

**For your lab evidence**, take screenshots of the player in two different locations showing that the visible circular region moves with the player, plus one screenshot with `H` showing the BFS path through the fog.

**TASK 3 — Timer Leaderboard** After the player solves the maze, save their time to a list (up to 5 entries, sorted ascending). Display the leaderboard on the win screen. Persist it in a JSON file called `leaderboard.json`.

### Important testing

Don't just check that the leaderboard appears.

Also check:

1. Finish maze.
2. Leaderboard appears.
3. Close game.
4. Start again.
5. Check that previous scores are still there.

You should see:

```
```text
leaderboard.json
```

Yes. For **Task 3**, add a persistent timer leaderboard without disturbing your existing `H` BFS hint or `R` maze regeneration.

The easiest implementation is to use Python's `time` module for the solve time and `json` for `leaderboard.json`.

1. Add these imports

At the top of `main.py` :

</> Python

```
import json
import os
```

```
import time
```

2. Add leaderboard variables/functions

Put these **above** your main game loop:

</> Python

```
LEADERBOARD_FILE = "leaderboard.json"
MAX_SCORES = 5

def load_leaderboard():
    if not os.path.exists(LEADERBOARD_FILE):
        return []

    try:
        with open(LEADERBOARD_FILE, "r") as file:
            scores = json.load(file)

            if isinstance(scores, list):
                return sorted(scores)[:MAX_SCORES]

    except (json.JSONDecodeError, OSError):
        pass

    return []
```

This gives you:

- maximum **5 scores**
- lowest time first
- persistent storage
- automatic creation of `leaderboard.json`

3. Load the leaderboard when the game starts

Before your main loop:

</> Python

```
leaderboard = load_leaderboard()
```

So when you restart the game, the old scores are loaded from:

</> Plain text

```
leaderboard.json
```

4. Start the timer

You need to start timing when the player starts playing.

For example, after creating the maze and player:

</> Python

```
start_time = time.time()
```

If your game has a start screen, ideally start the timer when the actual maze gameplay begins.

5. Detect when the player reaches the exit

You probably already have something like:

</> Python

```
if player == exit:  
    won = True
```

or:

</> Python

```
if player_rect.colliderect(exit_rect):  
    won = True
```

When the player wins, calculate the time:

</> Python

```
elapsed_time = time.time() - start_time  
  
leaderboard = add_score(elapsed_time)  
  
won = True
```

Important: only add the score once

Otherwise the game loop will add hundreds of scores while the win screen is displayed.

Use a flag:

</> Python

```
score_saved = False
```

Initialize it before the game loop.

Then:

</> Python

```
if player_reached_exit and not score_saved:
    elapsed_time = time.time() - start_time

    leaderboard = add_score(elapsed_time)

    score_saved = True
    won = True
```

6. Display the leaderboard on the win screen

When `won == True`, display something like:

</> Python

```
font = pygame.font.Font(None, 36)

title = font.render("Leaderboard", True, (255, 255, 255))
screen.blit(title, (100, 100))

for i, score in enumerate(leaderboard):
    text = font.render(
        f"{i + 1}. {score:.2f} seconds",
        True,
        (255, 255, 255)
    )

    screen.blit(text, (100, 150 + i * 40))
```

You'll get:

</> Plain text

Leaderboard

1. 8.42 seconds
2. 10.17 seconds
3. 12.63 seconds
4. 15.91 seconds
5. 20.44 seconds

7. Reset the timer when R creates a new maze

Your existing R code from Task 1 should become something like:

</> Python

```
elif event.key == pygame.K_r:
    maze = generate_maze()

    show_hint = False
    shortest_path = []

    start_time = time.time()

    won = False
    score_saved = False
```

This is important.

Otherwise the timer could continue counting from the previous maze.

8. Complete logic together

The important game-state section should look approximately like this:

</> Python

```
leaderboard = load_leaderboard()

start_time = time.time()
score_saved = False
won = False
```

Then your event handling:

</> Python

```
if event.type == pygame.KEYDOWN:

    if event.key == pygame.K_h:
        show_hint = not show_hint

        if show_hint:
            shortest_path = bfs_shortest_path(maze, start, exit)
        else:
            shortest_path = []

    elif event.key == pygame.K_r:
        maze = generate_maze()

        show_hint = False
        shortest_path = []

    start_time = time.time()
    score_saved = False
    won = False
```

And when the player reaches the exit:

</> Python

```
if player_reached_exit and not score_saved:
    elapsed_time = time.time() - start_time

    leaderboard = add_score(elapsed_time)

    score_saved = True
    won = True
```

9. Win screen

Your win screen can be:

</> Python

```
if won:
    screen.fill((30, 30, 30))

    title_font = pygame.font.Font(None, 60)
    score_font = pygame.font.Font(None, 40)
```

```
title = title_font.render("Maze Completed!", True, (255, 255, 255))
screen.blit(title, (100, 60))

time_text = score_font.render(
    f"Your Time: {elapsed_time:.2f} seconds",
    True,
    (255, 255, 255)
)
screen.blit(time_text, (100, 130))

leaderboard_title = score_font.render(
    "Leaderboard",
    True,
    (255, 255, 255))
```

10. What `leaderboard.json` should look like

After finishing a few games, the file should look approximately like:

</> JSON

```
[
    7.42,
    9.81,
    11.35,
    14.62,
    18.27
]
```

The important thing is that it remains sorted:

</> Plain text

```
fastest
↓
7.42
9.81
11.35
14.62
18.27
↑
slowest
```

And only the **best 5** are retained.

11. VERY IMPORTANT testing

Don't just test that the leaderboard appears.

Test 1 — First score

⌕ Plain text

```
Start game
↓
Solve maze
↓
Win screen appears
↓
Check leaderboard
```

You should see something like:

⌕ Plain text

```
Maze Completed!

Your Time: 12.43 seconds

Leaderboard

1. 12.43 seconds
```

Test 2 — Persistence

Now:

⌕ Plain text

```
Close game completely
↓
Start game again
↓
Check leaderboard.json
```

The previous score must still exist.

Then solve another maze:

⌕ Plain text

Solve maze

↓

Win screen

You should now have:

</> Plain text

Leaderboard

1. 10.21 seconds
2. 12.43 seconds

Test 3 — More than 5 scores

Play enough times to generate 6+ scores.

The leaderboard should **never contain more than 5 entries**.

For example:

</> Plain text

Leaderboard

1. 7.12 seconds
2. 8.53 seconds
3. 10.21 seconds
4. 12.43 seconds
5. 15.67 seconds

The sixth/worst score should be removed if it doesn't belong in the top 5.

12. Your repository should now contain

After Task 3:

</> Plain text

```
12_maze_runner/  
|  
├─ main.py  
├─ README.md  
├─ .gitignore  
└─ leaderboard.json
```

If `leaderboard.json` is generated by the game, **make sure it is not accidentally ignored by `.gitignore`**.

Then:

</> Bash

```
git add .
git commit -m "Add persistent timer leaderboard"
git push
```

For your lab evidence, take screenshots of **the win screen with the leaderboard**, `leaderboard.json` **in the repository**, and **the leaderboard still present after restarting the game**.

TASK 4 — Difficulty Levels Add a difficulty select screen before the maze generates. Show three buttons (Easy, Medium, Hard) with different COLS/ROWS values. Let the player click to select, then launch the maze with those dimensions.

Test all three

You need to test:

```
```text
Easy → 10 × 8
Medium → 15 × 13
Hard → 20 × 18
```

For **Task 4**, the cleanest approach is to add a **difficulty-selection screen before maze generation**. Keep your existing BFS ( H ), fog of war, timer/leaderboard, and R functionality intact.

The required mapping is:

Difficulty	COLS	ROWS
Easy	10	8
Medium	15	13
Hard	20	18

1. Add difficulty settings

Near the top of `main.py` :

Python

```
DIFFICULTIES = {
 "Easy": (10, 8),
 "Medium": (15, 13),
 "Hard": (20, 18)
}
```

## 2. Add a difficulty-selection screen

Put this above your main game loop:

Python

```
def difficulty_screen(screen):
 font = pygame.font.Font(None, 60)
 button_font = pygame.font.Font(None, 40)

 buttons = {
 "Easy": pygame.Rect(100, 250, 200, 70),
 "Medium": pygame.Rect(350, 250, 200, 70),
 "Hard": pygame.Rect(600, 250, 200, 70)
 }

 while True:
 screen.fill((30, 30, 30))

 title = font.render("Select Difficulty", True, (255, 255, 255))
 title_rect = title.get_rect(center=(450, 120))
 screen.blit(title, title_rect)

 for name, rect in buttons.items():
 pygame.draw.rect(screen, (70, 70, 70), rect)
 pygame.draw.rect(screen, (255, 255, 255), rect, 2)
```

## 3. Get the selected difficulty before generating the maze

Instead of immediately generating the maze when the program starts, do:

Python

```
difficulty = difficulty_screen(screen)
```

```
COLS, ROWS = DIFFICULTIES[difficulty]

maze = generate_maze(COLS, ROWS)
```

So the flow becomes:

#### </> Plain text

Start program



Difficulty screen



Easy	Medium	Hard
10 × 8	15 × 13	20 × 18



User clicks



Generate maze

## 4. Your maze generator must accept dimensions

If your current function is something like:

#### </> Python

```
def generate_maze():
```

change it to:

#### </> Python

```
def generate_maze(cols, rows):
```

Then use `cols` and `rows` inside the function instead of fixed values.

For example:

#### </> Python

```
def generate_maze(cols, rows):
 maze = [[1 for _ in range(cols)] for _ in range(rows)]
```

```
Your existing maze-generation logic

return maze
```

Don't replace your existing maze-generation algorithm with this example. Just make sure it uses the selected `cols` and `rows`.

## 5. Important: update your R key

Your existing Task 1/2/3 R functionality should regenerate the maze using the **currently selected difficulty**.

Use:

### </> Python

```
elif event.key == pygame.K_r:
 maze = generate_maze(COLS, ROWS)

 show_hint = False
 shortest_path = []

 start_time = time.time()
 score_saved = False
 won = False
```

This means:

### </> Plain text

```
Currently Easy
↓
Press R
↓
New 10 × 8 maze
```

and:

### </> Plain text

```
Currently Hard
↓
Press R
```



New 20 × 18 maze

## 6. Make sure the screen size works

If your maze cell size is calculated based on the maze dimensions, recalculate it after difficulty selection.

For example:

Python

```
CELL_SIZE = min(
 SCREEN_WIDTH // COLS,
 SCREEN_HEIGHT // ROWS
)
```

If your existing project already calculates `CELL_SIZE` differently, keep that calculation but make sure it happens **after**:

Python

```
COLS, ROWS = DIFFICULTIES[difficulty]
```

Otherwise all three difficulties may appear the same size or go outside the screen.

## 7. If your player/exit positions depend on dimensions

Make sure these are also recreated after generating the new maze.

For example:

Python

```
start = (1, 1)
exit = (ROWS - 2, COLS - 2)
```

Notice that maze coordinates are normally:

Plain text

```
(row, column)
```

SO:

```
</> Python
```

```
ROWS - 2
```

is used for the row and:

```
</> Python
```

```
COLS - 2
```

for the column.

Use whatever start/exit logic your existing project already has if it differs.

## 8. Complete startup flow

Your startup section should conceptually be:

```
</> Python
```

```
pygame.init()
```

```
screen = pygame.display.set_mode((SCREEN_WIDTH, SCREEN_HEIGHT))
```

```
pygame.display.set_caption("Maze Runner")
```

```
difficulty = difficulty_screen(screen)
```

```
COLS, ROWS = DIFFICULTIES[difficulty]
```

```
CELL_SIZE = min(
 SCREEN_WIDTH // COLS,
 SCREEN_HEIGHT // ROWS
)
```

```
maze = generate_maze(COLS, ROWS)
```

```
start = (1, 1)
```

```
exit = (ROWS - 2, COLS - 2)
```

```
player = start
```

Then your existing main game loop starts.

---

## 9. Test all three, exactly as required

Don't test only Easy.

### Test 1 — Easy

Start the program.

Click:

</> Plain text

Easy

Verify:

</> Plain text

COLS = 10

ROWS = 8

The resulting maze should be 10 × 8.

---

### Test 2 — Medium

Close/restart the game.

Click:

</> Plain text

Medium

Verify:

</> Plain text

COLS = 15

ROWS = 13

The maze should be  $15 \times 13$ .

---

### Test 3 — Hard

Close/restart again.

Click:

</> Plain text

Hard

Verify:

</> Plain text

COLS = 20

ROWS = 18

The maze should be  $20 \times 18$ .

---

## 10. Test that previous features still work

Because you've now implemented Tasks 1–4, your final testing should also verify:

</> Plain text

Difficulty screen

↓

Select difficulty

↓

Maze generated

↓

Move player

↓

Fog of War works

↓

H → BFS path appears

↓

H → BFS path disappears

↓

R → New maze of SAME difficulty

↓

Solve maze

↓

Timer recorded



Then commit:

**</> Bash**

```
git add .
git commit -m "Add difficulty levels"
git push
```

For your **Lab 4 screenshots**, I'd take one screenshot of the difficulty-selection screen and then **one screenshot each for Easy, Medium, and Hard**. That gives you direct evidence that all three required dimensions were implemented.

Final Code Review Review the current Maze Runner pr

Do not make any changes yet. First inspect the comp

1. Task 1 – Shortest Path Hint

- H key toggles the BFS shortest path
- Path is correctly calculated from start to exi
- Path is visually highlighted

2. Task 2 – Fog of War

- Only cells within radius 3 around the player a
- Remaining maze area is covered by fog

3. Task 3 – Timer Leaderboard

Yes — this is the **right final-review prompt** to give your Vibe Coding tool. I would make one small improvement: explicitly tell it to inspect the actual files rather than relying on the README or assumptions.

Use this exact prompt:

Final Code Review Prompt



## Final Code Review — Maze Runner

Review the **current Maze Runner project/codebase** against **every requirement in the README and SE Lab 4 – Vibe Coding instructions**.

**Do not make any changes to the code, files, README, or project structure yet.**

First, inspect the **complete codebase**, including:

- `main.py` and all Python source files
- `README.md`
- `requirements.txt`
- `leaderboard.json` if it exists
- Any other project/configuration files relevant to the application

Do not assume that a feature works just because the README says it does. Verify the implementation in the actual code.

Give me a clear verification report with **PASS, FAIL, or NEEDS TESTING** for every requirement.

## 1. Task 1 — Shortest Path Hint

Verify:

- Pressing H toggles the shortest-path hint on/off.
- The shortest path is calculated using **BFS**.
- BFS correctly calculates the path from the maze **start** to the **exit**.
- The path is visually highlighted on the maze.
- Pressing H a second time removes the highlighted path.
- Pressing R generates a new maze and the BFS path is recalculated correctly for the new maze.

## 2. Task 2 — Fog of War

Verify:

- A dark fog overlay covers the maze.
- Only cells within **radius 3 cells around the player** are visible.
- Cells outside this radius remain covered by fog.
- The visible region moves correctly as the player moves.
- The player remains visible.
- Fog of war does not prevent normal maze movement.
- The implementation uses an appropriate transparent Pygame surface with `SRCALPHA` and a transparent circular region, as required by the lab.

**Important:** Verify that the implementation corresponds to a radius of **3 cells**, not merely an arbitrary pixel radius. If the code uses a pixel radius, determine whether it actually corresponds to approximately 3 maze cells based on the current cell size.

### 3. Task 3 — Timer Leaderboard

Verify:

- The timer starts appropriately when a maze begins.
- The completion time is recorded when the player reaches the exit.
- A maximum of **5 scores** is maintained.
- Scores are sorted from **fastest to slowest**.
- The leaderboard is displayed on the win screen.
- Scores are saved to `leaderboard.json`.
- `leaderboard.json` is loaded when the game starts.
- Scores persist after closing and restarting the game.
- A new score is added only once per maze completion.
- `leaderboard.json` is not accidentally ignored by `.gitignore`.

### 4. Task 4 — Difficulty Selection

Verify:

- A difficulty-selection screen appears **before the maze starts**.
- The player can select the difficulty by clicking the corresponding button.
- **Easy** =  $10 \times 8$
- **Medium** =  $15 \times 13$
- **Hard** =  $20 \times 18$
- The selected dimensions are actually passed to the maze-generation logic.
- The generated maze dimensions match the selected difficulty.
- Pressing R generates another maze using the currently selected difficulty.

### 5. Existing Game Controls

Verify that:

- W movement works.
- A movement works.
- S movement works.
- D movement works.
- Arrow-key movement works:
  - Up
  - Down

- Left
- Right
- R generates a fresh maze.
- H still works correctly after using R.
- Fog of war still works after using R.
- Timer and leaderboard still work after using R.

## 6. Overall Stability

Verify:

- The game starts successfully.
- The game runs without Python/runtime errors.
- There are no obvious undefined variables, missing imports, broken function calls, or unreachable functionality.
- Existing functionality has not been broken by Tasks 1–4.
- Maze generation works for all three difficulty levels.
- The game can be completed successfully.
- The win screen appears correctly.

If something cannot be verified through static code inspection, mark it **NEEDS TESTING** instead of assuming it works.

## 7. Project Structure

Verify that the repository contains the appropriate files and that the structure is sensible.

Check at minimum:

```
main.py
README.md
requirements.txt
leaderboard.json
```

Also identify any other important project files that exist.

Verify that:

- `requirements.txt` contains all required external Python dependencies.
- Standard-library modules such as `json`, `os`, `time`, and `collections` are not incorrectly listed as pip dependencies.
- There are no unnecessary or suspicious files that should not be committed.
- `.gitignore` is appropriate.

- `leaderboard.json` is handled correctly for persistence.

## 8. README and Lab Requirements

Compare the implementation against the actual README and Lab 4 requirements.

Identify:

- Any requirement implemented in code but missing from the README.
- Any README claim that is not actually implemented.
- Any required evidence/testing that still needs to be performed.
- Any mismatch between the stated requirements and the current implementation.

## Required Output Format

Start with an overall status:

**Overall Status: PASS / FAIL / NEEDS TESTING**

Then provide a table:

Requirement	Status	Verification
Task 1 – BFS Hint	PASS/FAIL/NEEDS TESTING	Brief explanation
H toggles path	...	...
BFS start → exit	...	...
Path visualization	...	...
Task 2 – Fog of War	...	...
Radius 3 visibility	...	...
Fog overlay	...	...
Task 3 – Timer	...	...
Top 5 leaderboard	...	...
Sorted scores	...	...
Persistence	...	...
Task 4 – Difficulty	...	...

Easy 10×8	...	...
Medium 15×13	...	...
Hard 20×18	...	...
Movement controls	...	...
R key	...	...
Project structure	...	...
requirements.txt	...	...

After the table, provide these sections:

## Critical Issues

List only issues that must be fixed.

## Needs Testing

List functionality that looks correct from code inspection but requires actually running the game to verify.

## Minor Issues

List non-critical issues or cleanup suggestions.

## Final Submission Checklist

Give me a concise checklist of everything I should test, screenshot, commit, and push before submitting the lab.

**Important: DO NOT modify anything. This is a review-only step. Wait for my explicit instruction before making any fixes.**

This version is better because it catches an especially important issue: “radius 3” should mean 3 maze cells, not just a hard-coded pixel value. It also makes the tool distinguish between something it can prove from code and something that genuinely needs runtime testing.

## Final Code Review — Maze Runner

Review the **current Maze Runner project/codebase** against **every requirement in the README and SE Lab 4 – Vibe Coding instructions**.

**Do not make any changes to the code, files, README, or project structure yet.**

First, inspect the **complete codebase**, including:

- `main.py` and all Python source files
- `README.md`
- `requirements.txt`
- `leaderboard.json` if it exists
- Any other project/configuration files relevant to the application

Do not assume that a feature works just because the README says it does. Verify the implementation in the actual code.

Give me a clear verification report with **PASS, FAIL, or NEEDS TESTING** for every requirement.

## 1. Task 1 — Shortest Path Hint

Verify:

- Pressing `H` toggles the shortest-path hint on/off.
- The shortest path is calculated using **BFS**.
- BFS correctly calculates the path from the maze **start** to the **exit**.
- The path is visually highlighted on the maze.
- Pressing `H` a second time removes the highlighted path.
- Pressing `R` generates a new maze and the BFS path is recalculated correctly for the new maze.

## 2. Task 2 — Fog of War

Verify:

- A dark fog overlay covers the maze.
- Only cells within **radius 3 cells around the player** are visible.
- Cells outside this radius remain covered by fog.
- The visible region moves correctly as the player moves.
- The player remains visible.

- Fog of war does not prevent normal maze movement.
- The implementation uses an appropriate transparent Pygame surface with `SRCALPHA` and a transparent circular region, as required by the lab.

**Important:** Verify that the implementation corresponds to a radius of **3 cells**, not merely an arbitrary pixel radius. If the code uses a pixel radius, determine whether it actually corresponds to approximately 3 maze cells based on the current cell size.

### 3. Task 3 — Timer Leaderboard

Verify:

- The timer starts appropriately when a maze begins.
- The completion time is recorded when the player reaches the exit.
- A maximum of **5 scores** is maintained.
- Scores are sorted from **fastest to slowest**.
- The leaderboard is displayed on the win screen.
- Scores are saved to `leaderboard.json`.
- `leaderboard.json` is loaded when the game starts.
- Scores persist after closing and restarting the game.
- A new score is added only once per maze completion.
- `leaderboard.json` is not accidentally ignored by `.gitignore`.

### 4. Task 4 — Difficulty Selection

Verify:

- A difficulty-selection screen appears **before the maze starts**.
- The player can select the difficulty by clicking the corresponding button.
- **Easy** =  $10 \times 8$
- **Medium** =  $15 \times 13$
- **Hard** =  $20 \times 18$
- The selected dimensions are actually passed to the maze-generation logic.
- The generated maze dimensions match the selected difficulty.
- Pressing `R` generates another maze using the currently selected difficulty.

## 5. Existing Game Controls

Verify that:

- W movement works.
- A movement works.
- S movement works.
- D movement works.
- Arrow-key movement works:
  - Up
  - Down
  - Left
  - Right
- R generates a fresh maze.
- H still works correctly after using R .
- Fog of war still works after using R .
- Timer and leaderboard still work after using R .

## 6. Overall Stability

Verify:

- The game starts successfully.
- The game runs without Python/runtime errors.
- There are no obvious undefined variables, missing imports, broken function calls, or unreachable functionality.
- Existing functionality has not been broken by Tasks 1–4.
- Maze generation works for all three difficulty levels.
- The game can be completed successfully.
- The win screen appears correctly.

If something cannot be verified through static code inspection, mark it **NEEDS TESTING** instead of assuming it works.

## 7. Project Structure

Verify that the repository contains the appropriate files and that the structure is sensible.

Check at minimum:

```
main.py
README.md
requirements.txt
leaderboard.json
```

Also identify any other important project files that exist.

Verify that:

- `requirements.txt` contains all required external Python dependencies.
- Standard-library modules such as `json`, `os`, `time`, and `collections` are not incorrectly listed as pip dependencies.
- There are no unnecessary or suspicious files that should not be committed.
- `.gitignore` is appropriate.
- `leaderboard.json` is handled correctly for persistence.

## 8. README and Lab Requirements

Compare the implementation against the actual README and Lab 4 requirements.

Identify:

- Any requirement implemented in code but missing from the README.
- Any README claim that is not actually implemented.
- Any required evidence/testing that still needs to be performed.
- Any mismatch between the stated requirements and the current implementation.

## Required Output Format

Start with an overall status:

**Overall Status: PASS / FAIL / NEEDS TESTING**

Then provide a table:

Requirement	Status	Verification
Task 1 – BFS Hint	PASS/FAIL/NEEDS TESTING	Brief explanation
H toggles path	...	...
BFS start → exit	...	...
Path visualization	...	...
Task 2 – Fog of War	...	...
Radius 3 visibility	...	...
Fog overlay	...	...
Task 3 – Timer	...	...
Top 5 leaderboard	...	...
Sorted scores	...	...
Persistence	...	...
Task 4 – Difficulty	...	...
Easy 10×8	...	...
Medium 15×13	...	...
Hard 20×18	...	...
Movement controls	...	...
R key	...	...
Project structure	...	...
requirements.txt	...	...

After the table, provide these sections:

## Critical Issues

List only issues that must be fixed.

Needs Testing

List functionality that looks correct from code inspection but requires actually running the game to verify.

Minor Issues

List non-critical issues or cleanup suggestions.

Final Submission Checklist

Give me a concise checklist of everything I should test, screenshot, commit, and push before submitting the lab.

Important: DO NOT modify anything. This is a review-only step. Wait for my explicit instruction before making any fixes.

Worked for 2m 11s

I reviewed the **actual Maze Runner source files available in your project files**, the current README, requirements.txt , .gitignore , and the official Lab 4 handout. I also ran a **Python syntax/compile check** on the source files. I did **not modify anything**.

The Lab 4 handout confirms the required workflow: four tasks, before/after videos, updated code, exported chat history, and separate commits for each task.  Lab08\_Transaction\_Isolation\_Stu...

Overall Status: FAIL — 2 things need fixing before final submission

Most of the implementation is actually good. The two things I would **not submit yet** are:

- 1. Fog radius is slightly larger than the required 3 cells.
- 2. The leaderboard is not loaded into the game state when the game starts, even though it eventually loads correctly when a new score is added.

There are also a few things that still **NEED RUNTIME TESTING**, especially because Pygame isn't installed in my review environment.

Verification Report

Requirement	Status	Verification
Task 1 – BFS Hint	PASS	BFS implementation exists and is correctly integrated.
H toggles path	PASS	K_h toggles hint_on ; second press clears hint_path .

Requirement	Status	Verification
BFS start → exit	● PASS	BFS starts at (0,0) and ends at (rows-1, cols-1) , matching maze generation.
Path visualization	● PASS	Path cells are drawn as colored semi-transparent squares.
New BFS path after R	● PASS	<code>reset()</code> creates a new maze and clears the old hint.
<b>Task 2 – Fog of War</b>	● NEEDS FIX	Fog implementation is present, but radius calculation is not exactly 3 cells.
Radius 3 visibility	● FAIL	<code>FOG_RADIUS=3</code> , <code>CELL=40</code> , but radius becomes $3*40 + 20 = 140\text{px} = 3.5$ cells.
Fog overlay	● PASS	Uses <code>pygame.Surface(..., pygame.SRCALPHA)</code> and <code>pygame.draw.circle</code> .
Fog follows player	● PASS	Circle center uses <code>self.player.rect.centerx/centery</code> .
Player visible	● PASS	Player is inside the transparent circular cutout.
<b>Task 3 – Timer</b>	● PASS	Timer starts in <code>_start_maze()</code> and updates during play.
Completion time	● PASS	Time is calculated when player reaches exit.
Top 5 leaderboard	● PASS	Scores are sorted and sliced with <code>scores[:MAX_SCORES]</code> .
Sorted fastest → slowest	● PASS	<code>scores.sort()</code> is used.
Display after winning	● PASS	Win screen renders leaderboard entries.
JSON persistence mechanism	● PASS	<code>save_leaderboard()</code> writes <code>leaderboard.json</code> .
Load leaderboard at game start	● FAIL	<code>self.leaderboard = []</code> ; <code>load_leaderboard()</code> is not called in <code>__init__</code> .
Persistence after restart	● NEEDS TESTING	Code supports it, but actual close/restart test must be performed.

Requirement	Status	Verification
Score added only once	 PASS	State changes to "won" immediately after adding score, stopping further updates.
<b>Task 4 – Difficulty</b>	 PASS	Difficulty screen and dimension mapping are implemented.
Difficulty before maze	 PASS	Initial state is "difficulty" .
Easy 10×8	 PASS	("Easy", 10, 8) .
Medium 15×13	 PASS	("Medium", 15, 13) .
Hard 20×18	 PASS	("Hard", 20, 18) .
Click selection	 PASS	Mouse click is checked against each difficulty button.
Correct maze generation	 PASS	Selected cols, rows are passed to generate_maze() .
R same difficulty	 PASS	reset() calls _start_maze(self.cols, self.rows) .
W movement	 PASS	Implemented in Player.move() .
A movement	 PASS	Implemented in Player.move() .
S movement	 PASS	Implemented in Player.move() .
D movement	 PASS	Implemented in Player.move() .
Arrow keys	 PASS	Up/down/left/right are implemented.
R fresh maze	 PASS	generate_maze() is called again.
H after R	 PASS	Reset clears hint state.
Fog after R	 PASS	New player/maze is created and fog uses current player.
Timer after R	 PASS	New start_time is created.
Game starts without errors	 NEEDS TESTING	Syntax passes, but actual Pygame execution needs testing.

Requirement	Status	Verification
No obvious syntax errors	PASS	I ran Python compilation successfully.
Maze generation all sizes	PASS	Generator accepts arbitrary <code>cols</code> , <code>rows</code> ; runtime still needs testing.
Game can be completed	NEEDS TESTING	Must actually navigate and reach exit.
Win screen	PASS	"YOU WIN!" and leaderboard are implemented.
<code>requirements.txt</code>	PASS*	Contains only <code>pygame==2.5.2</code> , which is appropriate.
<code>.gitignore</code>	PASS	Ignores Python cache and virtual environments; doesn't ignore leaderboard.
<code>leaderboard.json</code>	NEEDS TESTING	It isn't currently present in the reviewed file snapshot, but code creates it after a win.
README accuracy	NEEDS MINOR UPDATE	README doesn't list <code>H</code> in the Controls table.
Project structure	NEEDS VERIFICATION	Code expects <code>game/game_engine.py</code> , <code>game/maze.py</code> , <code>game/player.py</code> ; available file snapshot doesn't expose <code>__init__.py</code> .

The implementation of BFS, leaderboard helpers, difficulty presets, and game state is visible in `game_engine.py` . `</>` `game_engine`

## 1. Task 1 — Shortest Path Hint

Status:  PASS

This part is implemented properly.

The BFS uses:

`</>` Python

```
queue = deque([start])
came_from = {start: None}
```

and explores four directions while checking the maze walls. It then reconstructs the path backwards and reverses it. `</> game_engine`

The toggle is also correct:

`</> Python`

```
self.hint_on = not self.hint_on
```

and:

`</> Python`

```
if self.hint_on:
 ...
else:
 self.hint_path = []
```

So:

`</> Plain text`

H → path appears  
H → path disappears

The path is rendered as colored squares using a transparent Pygame surface. `</> game_engine`

Also, R resets:

`</> Python`

```
self.hint_on = False
self.hint_path = []
```

so the old path isn't reused.

## Runtime test still required

You should still actually demonstrate:

`</> Plain text`

H → visible  
H → hidden

R → new maze

H → new path

## 2. Task 2 — Fog of War

Status: ● FAIL — radius needs correction

The good part first: the implementation follows the lab's required technique.

It creates:

**</> Python**

```
fog = pygame.Surface((width, height), pygame.SRCALPHA)
```

fills it with dark fog, and then uses:

**</> Python**

```
pygame.draw.circle(
 fog,
 (0, 0, 0, 0),
 (cx, cy),
 reveal_px
)
```

That is exactly the intended transparent-circle approach. </> game\_engine

### The problem

You have:

**</> Python**

```
CELL = 40
FOG_RADIUS = 3
```

but then:

**</> Python**

```
reveal_px = int(FOG_RADIUS * CELL + CELL // 2)
```

which gives:

⟨/⟩ Plain text

```
3 × 40 + 20
= 140 pixels
```

A radius of exactly **3 cells** should correspond to:

⟨/⟩ Plain text

```
3 × 40 = 120 pixels
```

So the current implementation is effectively using a **3.5-cell radius**.

**This is the one thing I would definitely fix.**

The intended calculation should be:

⟨/⟩ Python

```
reveal_px = FOG_RADIUS * CELL
```

Don't change it yet if you're still following your review-only instruction; just flag it for the Vibe Coding tool.

---

## 3. Task 3 — Timer Leaderboard

Mostly  **PASS**, but one implementation issue

The timer itself is correct:

⟨/⟩ Python

```
self.start_time = time.time()
```

and:

⟨/⟩ Python

```
self.elapsed = time.time() - self.start_time
```

When the player reaches the exit:

**</> Python**

```
self.leaderboard = add_score(self.elapsed)
self.won = True
self.state = "won"
```

So the completion time is recorded and the state changes to the win screen. </> game\_engine

The top-five logic is also correct:

**</> Python**

```
scores.append(round(elapsed, 2))
scores.sort()
scores = scores[:MAX_SCORES]
```

So:

**</> Plain text**

```
fastest
 ↓
1
2
3
4
5
 ↑
slowest
```

and anything beyond the top five is discarded.

## Persistence

The JSON functions are correctly implemented:

**</> Python**

```
with open(LEADERBOARD_FILE, "w") as f:
 json.dump(scores, f, indent=2)
```

and:

**</> Python**

```
with open(LEADERBOARD_FILE, "r") as f:
 data = json.load(f)
```

&lt;/&gt; game\_engine

## But there's a small issue

In `__init__()` :

&lt;/&gt; Python

```
self.leaderboard = []
```

There is **no**:

&lt;/&gt; Python

```
self.leaderboard = load_leaderboard()
```

So technically the requirement:

`leaderboard.json` is loaded when the game starts

is **not implemented**.

Interestingly, the leaderboard will still *appear persistent to the user after they win again*, because `add_score()` itself calls `load_leaderboard()` before adding the new score.

So the behavior may look correct, but the implementation isn't exactly what your review requirement asks for.

## Recommended fix

Eventually change:

&lt;/&gt; Python

```
self.leaderboard = []
```

to:

&lt;/&gt; Python

```
self.leaderboard = load_leaderboard()
```

---

## 4. Task 4 — Difficulty Selection

Status:  **PASS**

This is implemented nicely.

The exact presets are:

**</> Python**

```
DIFFICULTIES = [
 ("Easy", 10, 8),
 ("Medium", 15, 13),
 ("Hard", 20, 18),
]
```

**</> game\_engine**

The initial state is:

**</> Python**

```
self.state = "difficulty"
```

so the maze doesn't immediately start.

The user clicks a button and:

**</> Python**

```
self._start_maze(cols, rows)
```

generates the selected maze.

The maze generator accepts arbitrary dimensions and creates the corresponding grid. **</> maze**

So the required mapping is correct:

**</> Plain text**

Easy	→ 10 × 8
Medium	→ 15 × 13
Hard	→ 20 × 18

---

## 5. Movement

Status:  **PASS by code inspection**

Your `Player.move()` supports:

 **Python**

```
LEFT / A
RIGHT / D
UP / W
DOWN / S
```

and collision checking is implemented against the maze walls.  `player`

So all eight required movement inputs are present.

However, **actually moving through a maze needs runtime testing.**

---

## 6. R key

Status:  **PASS**

During gameplay:

 **Python**

```
if event.key == pygame.K_r:
 self.reset()
```

and:

 **Python**

```
def reset(self):
 self._start_maze(self.cols, self.rows)
```

So pressing R generates a fresh maze with the **same selected dimensions**.  `game_engine`

It also resets:

- player
- timer
- BFS path
- hint state
- maze

which is exactly what you want.

---

## 7. Project Structure

Your README expects:

### </> Plain text

```
maze-runner/
├─ main.py
├─ requirements.txt
├─ game/
│ ├── __init__.py
│ ├── game_engine.py
│ ├── maze.py
│ └── player.py
└─ README.md
```

 README(20261007-063756)

The code itself expects:

### </> Python

```
from game.maze import ...
from game.player import ...
```

and:

### </> Python

```
from game.game_engine import GameEngine
```

</> game\_engine      </> main(1)

## One thing to check on your actual GitHub repository

The Library snapshot shows:

### </> Plain text

```
main(1).py
requirements(1).txt
```

```
README(20261007-063756).md
.gitignore(1)
```

Those suffixes may simply be **Library duplicate-name handling**, so I would **not automatically treat them as actual GitHub filenames**.

But on GitHub, make absolutely sure you have:

</> Plain text

```
main.py
requirements.txt
README.md
game/
 game_engine.py
 maze.py
 player.py
```

and preferably:

</> Plain text

```
game/__init__.py
```

## 8. requirements.txt

Status: ● PASS

Your requirements file contains:

</> Plain text

```
pygame==2.5.2
```

 requirements(1)

That's correct.

You are **not** incorrectly listing:

</> Plain text

```
json
os
```

```
time
collections
```

because those are Python standard-library modules.

## 9. .gitignore

Status: ● PASS

Current ignore rules are:

</> Plain text

```
__pycache__/
*.pyc
.venv/
venv/
```

 .gitignore(1)

Importantly, it does **not** ignore:

</> Plain text

```
leaderboard.json
```

which is good for your persistence requirement if you intend to commit the generated file.

## 10. README

Status: ● NEEDS MINOR UPDATE

Your README correctly documents all four tasks and the required dimensions.

 README(20261007-063756)

However, the Controls table currently has:

</> Plain text

```
W / UP
S / DOWN
A / LEFT
```

```
D / RIGHT
R
```

but does **not explicitly** list H.

Since H is now an actual game control, I'd add:

```
</> Plain text
```

```
| H | Show/hide shortest path hint |
```

This is not a major problem, but it would make your README match the final implementation better.

---

## Critical Issues

### ● 1. Fix fog radius

Current:

```
</> Python
```

```
reveal_px = int(FOG_RADIUS * CELL + CELL // 2)
```

With CELL = 40 :

```
</> Plain text
```

```
140 px = 3.5 cells
```

Required:

```
</> Plain text
```

```
120 px = 3 cells
```

So this needs to become:

```
</> Python
```

```
reveal_px = FOG_RADIUS * CELL
```

## ● 2. Load leaderboard when game starts

Current:

Python

```
self.leaderboard = []
```

Recommended:

Python

```
self.leaderboard = load_leaderboard()
```

The existing persistence mechanism itself is good, but this makes the implementation satisfy the requirement explicitly.

---

## Needs Testing

These should **not** be marked as passed in your final report until you actually test them:

### Task 1

- ☐ H shows BFS path
- ☐ H hides BFS path
- ☐ R creates new maze
- ☐ H calculates new path

### Task 2

- ☐ Move around
- ☐ Only nearby area is visible
- ☐ Fog follows player
- ☐ Player remains visible
- ☐ Movement remains functional

### Task 3

- ☐ Complete maze
- ☐ Win screen appears
- ☐ Time appears
- ☐ Leaderboard appears
- ☐ Close game
- ☐ Restart

- ☐ Previous scores remain
- ☐ Generate more than 5 scores and verify only 5 remain

## Task 4

Test all three:

Plain text

Easy → 10 × 8  
Medium → 15 × 13  
Hard → 20 × 18

The Lab handout specifically requires you to run the Python game, record before/after videos, use Vibe Coding prompts, and maintain separate commits for each task.  Lab08\_Transaction\_Isolation\_Stu...

## Minor Issues

1. README doesn't list H in Controls.
2. `dim_labels` in `_draw_difficulty_screen()` is created but never used:

Python

```
dim_labels = [...]
```

Not harmful.

3. The `MOUSEBUTTONDOWN` code contains this odd unused check:

Python

```
if event.key if hasattr(event, "key") else False:
```

It doesn't break the main functionality, but it's unnecessary.

4. The Easy win-screen panel is taller than the maze display area, so its top can extend partly off-screen. This is a UI polish issue, not a core requirement failure.
5. `leaderboard.json` isn't present in the reviewed file snapshot yet. That's okay **if it gets generated and committed after you actually complete a maze**, but you need to verify it.

## One important thing I verified

I ran a Python compilation check on:

**</> Plain text**

```
main.py
game/game_engine.py
game/maze.py
game/player.py
```

and there were **no syntax errors**.

However, I could not perform a genuine Pygame runtime test in this review environment because Pygame itself isn't installed here. So I am deliberately marking the interactive behavior **NEEDS TESTING**, rather than falsely calling it passed.

---

## Final Submission Checklist

Before you submit, I would do this exact sequence:

### Fix first

- ☐ Correct fog radius to exactly 3 cells.
- ☐ Load leaderboard at startup.
- ☐ Add H to README Controls.
- ☐ Confirm actual GitHub filenames are `main.py` , `README.md` , `requirements.txt` .

### Runtime testing

- ☐ Easy → **10×8**
- ☐ Medium → **15×13**
- ☐ Hard → **20×18**
- ☐ W/A/S/D movement
- ☐ Arrow movement
- ☐ H on/off
- ☐ R regeneration
- ☐ Fog follows player
- ☐ Complete maze
- ☐ Timer recorded
- ☐ Leaderboard displayed
- ☐ Close game
- ☐ Restart
- ☐ Previous leaderboard remains
- ☐ Verify `leaderboard.json`
- ☐ Verify top 5 behavior

## Evidence for Lab 4

The official handout requires:

- ☐ 10-second video before changes
- ☐ 10-second video after changes
- ☐ Updated code
- ☐ Exported ChatGPT/Vibe Coding chat as PDF/doc
- ☐ Separate commit for each task
- ☐ Push to your personal repository
- ☐ No PR to the original SETAPESU26 repository.

 Lab08\_Transaction\_Isolation\_Stu...

## Git commits

You should have separate commits along the lines of:

**</> Bash**

```
git log --oneline
```

showing:

**</> Plain text**

```
Add BFS shortest path hint
Add fog of war
Add persistent timer leaderboard
Add difficulty levels
```

Then push the final version.

**Bottom line:** you're quite close. I would **not redo the project**. Fix those two concrete code issues, update the README control line, then do the runtime tests. After that, your implementation should be in very good shape for Lab 4.